DSA & CS Fundamentals

Pen-and-paper C++ code and core theory, basics only

Nirmalya Mandal - SAP Labs Interview Prep

Study pack - part 4 of 8

Contents

How to use this

This covers exactly the topics your seniors reported and nothing fancier - basics done correctly. The interview is **pen and paper**, and the seniors said two things that matter most:

- **Explain your logic out loud while you write.** Correct logic beats perfect syntax; they forgive small syntax slips if the reasoning is right.
- **Write in the language you're comfortable with.** For you that's **C++** for data structures.

For each topic: the idea in one breath, the C++ code kept short and clean, and the **"say this aloud"** line to narrate while writing. Practise writing these by hand at least once - muscle memory helps under pressure.

Big-O in one minute (so you can talk about efficiency)

Big-O describes **how the running time grows as the input grows**. Know these five:

- **O(1)** - constant. Same time regardless of size. (Hashmap lookup.)
- **O(log n)** - logarithmic. Halves the problem each step. (Binary search.)
- **O(n)** - linear. Look at each item once. (Linear search, one pass.)
- **O(n log n)** - the good sorting speed. (Merge sort, `std::sort`.)
- **O(n^2)** - quadratic. Nested loops over the data. (Bubble/insertion sort.)

Say aloud: "This runs in O(…) because for each of the n items I do (constant / another loop / half the work)."

Searching

Linear search - O(n)

Walk through the array, return the index if found.

```
int linearSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) return i;
    }
    return -1;  // not found
}
```

Say aloud: "I check each element one by one; if it matches I return its index, else -1. O(n)."

Binary search - O(log n) [ASKED]

Works **only on a sorted array**. Check the middle; if the target is smaller, search the left half, else the right half. Half the search space disappears each step.

```
int binarySearch(int arr[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;  // not found
}
```

Say aloud: "The array is sorted, so I look at the middle. If it's the target I'm done. If the target is bigger I discard the left half by moving low up; if smaller I discard the right half. Each step halves the range, so O(log n)."

Two things to mention if asked: I write `mid = low + (high - low) / 2` instead of `(low + high) / 2` to avoid integer overflow on large indices. And the loop condition is `low <= high` so a single remaining element still gets checked.

Floor and ceiling with binary search [ASKED]

In a sorted array, given x:

- **Floor(x)** = the largest element **<= x**.
- **Ceiling(x)** = the smallest element **>= x**.

```
int floorIdx(int arr[], int n, int x) {
    int low = 0, high = n - 1, ans = -1;
```

```
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] <= x) {   // candidate, look right
            ans = mid;
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return ans;  // index of floor, or -1
}

int ceilIdx(int arr[], int n, int x) {
    int low = 0, high = n - 1, ans = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] >= x) {   // candidate, look left
            ans = mid;
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }
    return ans;  // index of ceiling, or -1
}
```

Say aloud: "For floor, whenever an element is <= x it's a possible answer, so I record it and search right for something even closer. For ceiling I do the mirror: record when >= x and search left. Still O(log n)."

Sorting

You almost never need to hand-write a fast sort in an interview - if asked to sort, you can say "in real code I'd use `std::sort`, which is O(n log n)," and then write a simple one if they want to see it. Know **one** simple sort well.

Bubble sort - O(n^2), easy to write

Repeatedly swap adjacent out-of-order pairs; the largest "bubbles" to the end each pass.

```
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;  // already sorted
    }
}
```

Say aloud: "Each pass pushes the largest remaining element to the end by swapping neighbours. The `swapped` flag lets me stop early if a pass makes no swaps. O(n^2)."

What to know about the others (no need to write)

- **Insertion sort** - build the sorted part one element at a time; good for nearly-sorted data. O(n^2).
- **Selection sort** - repeatedly pick the minimum and place it. O(n^2).
- **Merge sort / Quick sort** - the fast ones, O(n log n). `std::sort` uses an optimised hybrid. Mention these to show you know the difference between O(n^2) and O(n log n).

Hashmaps [ASKED]

The idea

A hashmap stores **key-value pairs** and gives **O(1) average** lookup, insert, and delete. A **hash function** turns the key into an array index, so you jump straight to the value instead of searching.

- **Collisions** (two keys landing on the same index) are handled by **chaining** (a small list at each slot) or **open addressing** (probe for the next free slot).
- Worst case is O(n) if everything collides, but with a good hash function the average is O(1).
- In **C++** it's `unordered_map`. (`map` is different - it's a sorted tree, O(log n).) In **JavaScript** it's an object or `Map`.

C++ usage

```cpp
#include <unordered_map>
#include <string>
using namespace std;

unordered_map<string, int> age;
age["Nirmalya"] = 22;      // insert
age["Amit"] = 25;

if (age.count("Amit")) {   // check exists
    int a = age["Amit"];   // lookup, O(1) avg
}
age.erase("Amit");          // delete
```

The classic hashmap problem: count frequencies

```cpp
unordered_map<int, int> freq;
for (int i = 0; i < n; i++) {
    freq[arr[i]]++;   // count each number
}
```

Say aloud: "A hashmap hashes the key to an index for O(1) average access. Here I use one to count how many times each number appears in a single pass. If two keys collide, chaining or probing resolves it."

Interview tip: many "do this fast" problems are secretly hashmap problems - "find if two numbers sum to a target," "find the first duplicate." If they ask you to speed up an O(n^2) brute force, think **hashmap** first.

Linked lists

The idea

A linked list is a chain of **nodes**, each holding **data** and a **pointer** to the next node. Unlike an array, it isn't stored in one contiguous block, so inserting and deleting are cheap (just rewire pointers) but random access is slow (you must walk from the start).

- **Singly linked:** each node points to `next` only.
- **Doubly linked:** each node points to both `next` and `prev` - you can walk both ways.

Doubly linked list - insert and delete [ASKED]

This is the one the seniors were asked. Node definition first:

```
struct Node {
    int data;
    Node* prev;
    Node* next;
    Node(int d) {
        data = d;
        prev = nullptr;
        next = nullptr;
    }
};
```

Insert at the front

```
Node* insertFront(Node* head, int value) {
    Node* node = new Node(value);
    node->next = head;
    if (head != nullptr) {
        head->prev = node;
    }
    return node;  // new head
}
```

Say aloud: "I make a new node, point its next at the current head, and if the list wasn't empty I point the old head's prev back at the new node. The new node becomes the head."

Insert at the end

```
Node* insertEnd(Node* head, int value) {
    Node* node = new Node(value);
    if (head == nullptr) return node;
    Node* cur = head;
```

```
    while (cur->next != nullptr) {
        cur = cur->next;      // walk to last node
    }
    cur->next = node;
    node->prev = cur;
    return head;
}
```

Say aloud: "If the list is empty the new node is the head. Otherwise I walk to the last node, link its next to the new node, and set the new node's prev back to it."

Delete a node by value

```
Node* deleteNode(Node* head, int value) {
    Node* cur = head;
    while (cur != nullptr && cur->data != value) {
        cur = cur->next;      // find the node
    }
    if (cur == nullptr) return head;  // not found

    if (cur->prev != nullptr)
        cur->prev->next = cur->next;  // bypass forward
    else
        head = cur->next;             // deleting head

    if (cur->next != nullptr)
        cur->next->prev = cur->prev;  // bypass backward

    delete cur;               // free memory
    return head;
}
```

Say aloud: "I walk to the node with that value. To remove it I connect its previous node's next to its next, and its next node's prev back to its previous - so nothing points at it. I handle the edge case where it's the head. Then I free it."

The whole trick with linked lists is pointer rewiring and edge cases (empty list, head, tail). Narrate each pointer change - that's what impresses.

Graphs: BFS and DFS [ASKED]

The setup

A graph is nodes (vertices) connected by edges. The easy way to store it is an **adjacency list** - for each node, a list of its neighbours. Both traversals need a `visited` array so you don't loop forever on cycles.

```
#include <vector>
#include <queue>
using namespace std;

// adj[u] = list of neighbours of u
vector<int> adj[100];
bool visited[100];
```

BFS - breadth-first search (uses a queue)

Explores **level by level**: all neighbours first, then their neighbours. Great for shortest path in an unweighted graph.

```
void bfs(int start) {
    queue<int> q;
    visited[start] = true;
    q.push(start);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        // process u here (e.g. print)
        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}
```

Say aloud: "BFS uses a queue. I mark the start visited and push it. Then I keep taking the front node, and for each unvisited neighbour I mark it visited and push it. Because a queue is first-in-first-out, I explore level by level."

DFS - depth-first search (recursion, i.e. a stack)

Goes **as deep as possible** down one path before backtracking.

```
void dfs(int u) {
    visited[u] = true;
    // process u here (e.g. print)
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v);           // recurse deeper
        }
    }
}
```

Say aloud: "DFS marks the current node visited, then recursively dives into each unvisited neighbour, going deep before coming back. Recursion uses the call stack, so it's naturally last-in-first-out."

One-line contrast to say: "BFS is level-by-level with a queue and finds shortest paths in unweighted graphs; DFS goes deep with a stack/recursion and is natural for things like cycle detection and exploring all paths."

DBMS - the theory they love

ACID properties [ASKED - and it's your DokLink story]

ACID is the four guarantees a **transaction** (a group of database operations treated as one unit) provides. Explain each with one line; you have a real example.

- **A - Atomicity:** all or nothing. Every step in the transaction succeeds, or none do. (In DokLink, reserving a bed either fully happens or fully rolls back - no half-booking.)
- **C - Consistency:** the database moves from one valid state to another, never breaking its rules. (A bed count never goes negative.)
- **I - Isolation:** concurrent transactions don't step on each other; the result is as if they ran one at a time. (This is exactly what stopped two users booking the last bed at once.)
- **D - Durability:** once committed, the data survives even a crash or power loss. (A confirmed booking stays confirmed.)

Say aloud: "ACID is Atomicity, Consistency, Isolation, Durability. I relied on all four in DokLink's bed booking - especially Atomicity and Isolation, which is how I solved the race condition where two people try to book the last ICU bed at the same instant."

Normalization (basics)

Normalization is organising tables to **reduce duplicate data and avoid update problems.** The gist:

- **1NF:** each cell holds a single value; no repeating groups.
- **2NF:** 1NF, and every non-key column depends on the **whole** primary key.
- **3NF:** 2NF, and no non-key column depends on another non-key column.

Say aloud: "Normalization removes redundancy by splitting data into related tables, so I don't store the same fact in two places and risk them disagreeing." You don't need more than this for the interview.

Primary key vs foreign key

- **Primary key:** uniquely identifies each row in a table (e.g. `user_id`).
- **Foreign key:** a column that points to another table's primary key, creating a relationship (e.g. an order's `user_id` links to the users table).

SQL vs NoSQL (quick)

- **SQL (PostgreSQL):** structured tables, fixed schema, strong consistency, transactions. Best when data is relational and correctness matters.
- **NoSQL (MongoDB):** flexible documents, easy to scale horizontally, good for unstructured or fast-changing data.

Scenario-based DB design [ASKED] - a worked example

They'll describe a business and ask you to design the tables. **Use a familiar example - your own DokLink.** The method:

1. **Find the entities** (the "things"): Users, Hospitals, Beds, Reservations, Payments.
2. **Give each a table with a primary key.**
3. **Connect them with foreign keys.**
4. **Decide the relationship type** (one-to-many, many-to-many).

A simple design:

Table	Key columns	Notes
Users	user_id (PK), name, phone	one user has many reservations
Hospitals	hospital_id (PK), name, location	one hospital has many beds
Beds	bed_id (PK), hospital_id (FK), type, status	type = general/ICU; status = free/reserved
Reservations	reservation_id (PK), user_id (FK), bed_id (FK), expires_at, status	links a user to a bed
Payments	payment_id (PK), reservation_id (FK), amount, status	one payment per reservation

Relationships to say aloud:

- A **Hospital** has many **Beds** - one-to-many (the Bed table carries `hospital_id`).
- A **User** has many **Reservations** - one-to-many.
- A **Reservation** links one **User** to one **Bed**, and has one **Payment**.

Say aloud while drawing: "First I list the entities. Each becomes a table with a primary key. Then I connect them with foreign keys - a bed belongs to a hospital, so the bed table stores the hospital's id. A reservation ties a user to a bed, so it stores both ids. That gives me clean one-to-many relationships and no duplicated data."

For a many-to-many (say, students and courses), explain you'd add a **junction table** (`enrollments` with `student_id` and `course_id`) - a good bonus point if the scenario needs it.

OOP - object-oriented programming

The four pillars (know all four)

- **Encapsulation:** bundle data and the methods that act on it inside a class, and hide the internals behind a clean interface (private fields, public methods).
- **Abstraction:** expose only what matters, hide the complexity. You use a `Car.drive()` without knowing the engine internals.
- **Inheritance:** a child class reuses and extends a parent class - an "is-a" relationship. [ASKED]
- **Polymorphism:** the same method call behaves differently depending on the actual object - "many forms." A `draw()` call works on a Circle or a Square, each drawing itself.

Inheritance - C++ example [ASKED]

```cpp
#include <iostream>
#include <string>
using namespace std;

class Vehicle {                 // parent / base
public:
    string brand = "Generic";
    void honk() {
        cout << "Beep!" << endl;
    }
};

class Car : public Vehicle {    // child inherits Vehicle
public:
    int wheels = 4;
};

int main() {
    Car c;
    c.honk();                   // inherited from Vehicle
    cout << c.brand << endl;    // inherited field
    cout << c.wheels << endl;   // its own field
    return 0;
}
```

Say aloud: "Car inherits from Vehicle, so it automatically gets Vehicle's `honk()` method and `brand` field, and adds its own `wheels`. It models 'a Car is a Vehicle' and avoids rewriting shared code."

Polymorphism - one line of code to recognise

Achieved with **virtual functions** in C++: a base-class pointer calls the derived class's version at runtime (**method overriding**). Also **overloading** - same function name, different parameters. You

don't need to write it, just recognise the terms.

Exception handling [ASKED]

The idea

A structured way to handle runtime errors without crashing: put risky code in **`try`**, and if it throws an error, **`catch`** and handle it. This keeps the program stable and the failure explicit.

C++

```cpp
#include <iostream>
using namespace std;

int divide(int a, int b) {
    if (b == 0) {
        throw runtime_error("divide by zero");
    }
    return a / b;
}

int main() {
    try {
        cout << divide(10, 0) << endl;
    } catch (const exception& e) {
        cout << "Error: " << e.what() << endl;
    }
    return 0;
}
```

Say aloud: "I put the risky call in a try block. If it throws, control jumps to catch, where I handle the error gracefully instead of the program crashing. Here I throw when dividing by zero and catch it to print a clean message."

The same idea in other languages (mention if relevant)

- **Python:** `try: ... except SomeError as e: ...`, with optional `finally:`.
- **JavaScript:** `try { ... } catch (e) { ... } finally { ... }`.

One line to add: "`finally` runs whether or not an error happened - I use it to clean up resources like closing a file or a database connection."

Final DSA checklist

Be able to write these by hand, narrating as you go:

1. **Binary search** (and floor/ceiling variant).
2. **Doubly linked list** - insert front/end, delete by value.
3. **BFS** (queue) and **DFS** (recursion).
4. **Hashmap** frequency count with `unordered_map`.
5. **One simple sort** (bubble) + know which sorts are O(n log n).

And be able to *explain*, not write:

6. **ACID** (with the DokLink tie-in), **normalization** basics, **scenario DB design** method.
7. **Inheritance** + the **four OOP pillars**.
8. **Exception handling** with try/catch.

Write each of the code ones on paper once tonight. When it flows from your hand, you're ready.